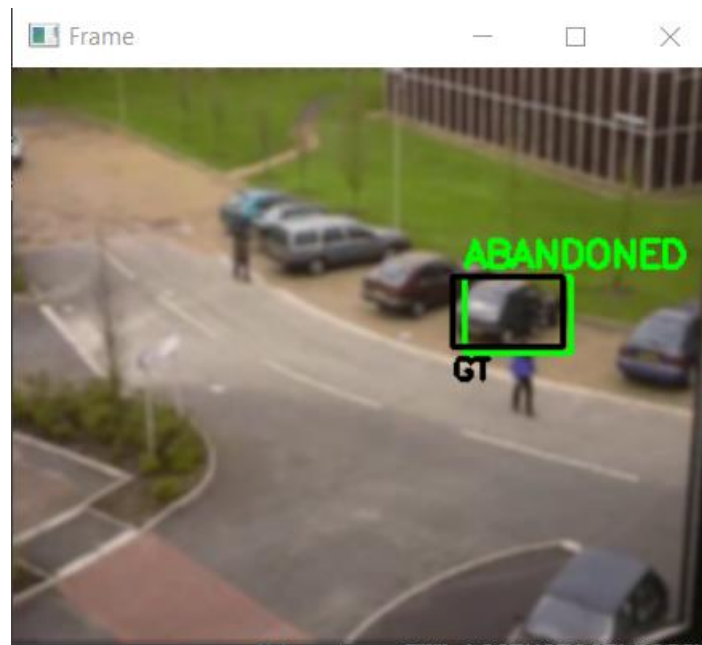


Computer Vision Assignment

Oran O'Sullivan – 22336707

Introduction:

The problem posed of classifying removed and abandoned objects is a nuanced one without a fixed concrete solution. While my attempt accomplishes the proposed task reasonably well, it certainly isn't perfect. Throughout this report I will outline my approach, I will discuss the merits and shortcomings of my design, and I will highlight what I would do differently.



Theory of Processing Done:

Since the assignment itself was divided up into two sections it makes sense to do the same when illustrating my solution.

Task 1 - Static Object Detection:

At a high level, I compare a background model to a motion mask, get the difference and use this to highlight potential static areas. There is quite a lot of depth behind what sounds like a simplistic and straightforward approach. The first major decision I encountered was what technique to create my background model with. I chose ultimately to use **GMM**. While the other techniques yielded positive results GMM was far and away the most robust. I also added some **Opening** and **Closing**, to help reduce the impact of noise. It's also worth noting that I chose to **disable shadow detection** for the GMM

mask, as it again helped reduce noise. I specified a slow learning rate for the GMM mask, to keep the stationary object from fading into the background immediately.



The **GMM** alone was not enough to identify stationary regions. I also required a **motion mask**. This **motion mask** compared the current frame to previous frames, highlighting any differences between them. What shows up here is any moving objects in the foreground. I decided to implement a **buffer** of 5 frames. This helped detect some of the slower more subtle movements. Before performing subtraction, I converted the blurred colour image to **greyscale**. From here a subtraction was made between the current frame and the previous 5 frames. The output of this **subtraction** is then **Thresholded** to deliver the final motion mask.

From here the binary motion mask can be compared to the binary GMM mask. I performed a **bitwise AND** between the GMM mask and the **inverse** of the motion mask. In other words, if an object appears the foreground but is stationary, it is a static object.

In theory this would be enough, however reality is rarely this straight forward. Things like lighting changes and people moving unpredictably can cause undesirable results. One thing I implemented was a **persistence mask**. The basic idea was drawing a bounding box around the static object in the stationary mask, and if it stayed long enough it was classified as a static object. I used a persistence threshold of 4 seconds. This was based off what worked well for the videos but is something that could be tweaked to suit a specific application.

Task 2 - Static Object Classification:

The high-level Idea behind my classification approach was to use the area identified in the static object detection. Then compare the contours in this area in the current frame, with those of a previous frame. Significant contour overlap suggests object removal and the inverse suggested object abandonment. This was considered (x1) as was edge count (x0.3), to help remove uncertainty. Persistence of +5 votes across +15 frames resulted in a positive identification.

The first part of the classification system is the **Baseline History**. This is an array of 50 frames taken from the beginning of the video. Technically this wouldn't work "live" but it's close enough to real time to be used in this scenario.

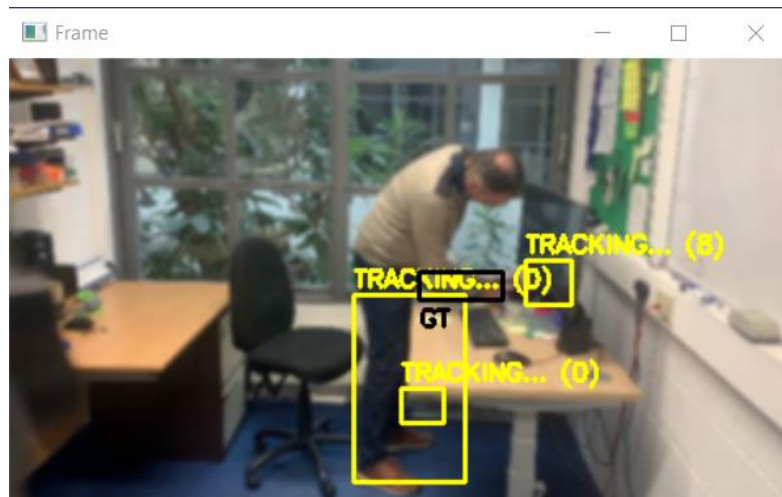
Once the baseline has been established, a "safe" **bounding box** is drawn around the static object detected. Safe simply refers to the fact that there is a check in place insuring the box is not out of image range. **Canny Edge Detection** is used on the identified stationary region. I chose canny because it's the only edge detection technique that produces a binary image as an output (without the help of other techniques).



Before using **Canny** on the historic frames, they are converted to greyscale, and the region within the **safe box** (mentioned previously) is isolated. A **Bitwise AND** operation is used to calculate the overlap between the current frame and the historic frame. The output of the AND operation is compared two thresholds. One for abandoned (< 0.1) and one for removed (> 0.25). Depending on the result a score is kept.

This process is repeated for each of the 4 frames in the historic baseline. The scores are the compared. Also, part of the classification process is a raw **edge count**. Early on I noticed removed objects were rarely being classified correctly. To counteract this, I employed an edge count comparison. This metric is not reliable and is thus weighted by a factor of 0.3 and taken into account for the classification decision. At least 3 comparisons need to be made for a reliable classification. Once all the baseline frames have been compared, the object is given a vote for the decided class.

The code gave reasonable results up to this point however, I had a major issue with false positives. Since there was no duration threshold in place, stationary objects were classified immediately. This caused an abundance of false positives. To combat this, I added a **tracking system**.



When an object is initially detected in the persistence mask, it needs to be within the area bounds and not be too elongated. If the object is new, a **tracked object** struct was created. This object was a way to track whether an object has enough consistency to be considered for classification. If the object has been seen in previous frames (threshold is 15) it can be considered for classification. While this approach is computationally heavy, it massively reduced the number of false positive classifications from the thousands to single digits.

Results and Analysis:

Immediate observations:

- The solution does not work for video 10
- There are still some false positives
- There are some misclassifications
- It's computationally heavy
- It can be affected by light

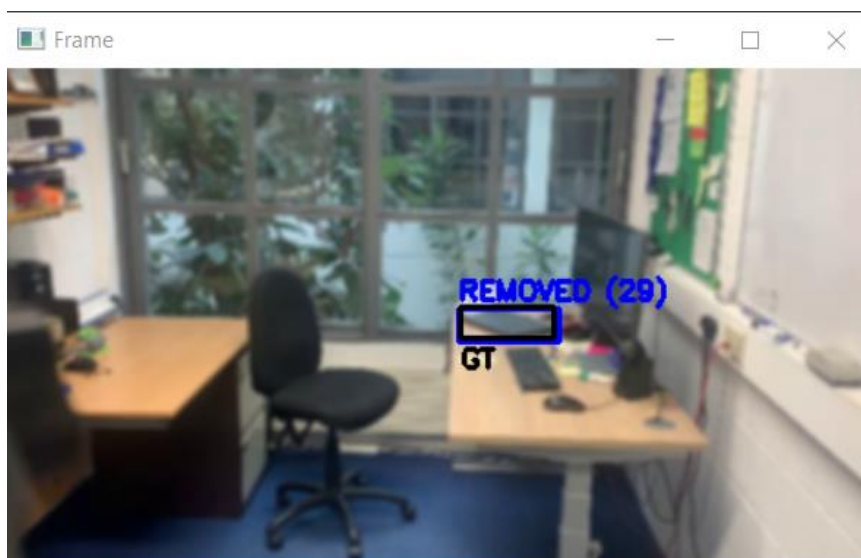
Visual analysis:

There were a couple of things immediately obvious from watching the visual output from the program. The first is that it worked quite well more often than not; the program was able to correctly identify the stationary regions. The first two videos appear to work well, despite several "incorrect areas being identified". I would argue the ground truth is incorrect in this scenario. I can understand why a person standing still in the scene for a while isn't labelled stationary, however I would make the argument that reflections should be accounted for in the ground truth. In both video 1 and video 2, the change in the scene causes the reflection in the monitor to change. My solution correctly labels them as stationary affected areas, conflicting with the ground truth that doesn't account for this at all. In a more specialised solution perhaps it would be worth while,

however for the purposes of this application I would make the argument the ground truth should be more robust to reflect the range of applications of the program.



The next is that the neither the classification or the stationary object detection is perfect. In video 2 the abandoned object is mistakenly labelled as removed. This may have been due to the person partially standing in front of the object, causing the classification system some confusion. It may also have been caused by a slight bias towards removal in the solution. In video 5 the abandoned object is not detected as it is too small. I made a conscious decision to ignore objects smaller than 150. This was to help eliminate noise within my solution. I compared the success in this one video against failures within others, when I lowered the threshold and ultimately decided that it was a better decision to leave the low area threshold at 150.



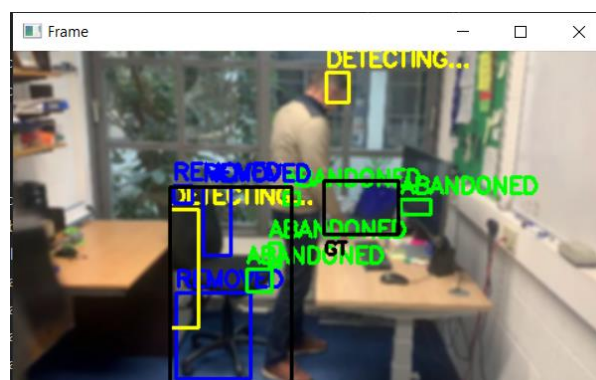
One shortcoming of my application was the inability to detect the second event in any of the videos. For example, it detected when the bag was abandoned in video 6, yet it failed to detect when it was removed. This was an issue I encountered throughout the

assignment and ran out of time to solve it. Since the learning rate was so low to allow the objects more time to stabilise, this also means it takes longer for them to integrate into the background.

Another thing that became obvious is that people standing still had a tendency to interfere with the program. If they were there long enough and stayed still enough, my application thought they were a stationary object. While I certainly tuned my program to lower this error over the course of the assignment, I was not able to eliminate it entirely.



The final major observation was the complete failure of video 10. The first reason it was such a failure is that I failed to develop a mechanism to classify “other” events. In earlier versions this resulted in copious incorrect classifications. However, in the final version there were counter measures in place to mitigate the damage. Yet video 10 still found a way to upset my solution. My code is on the heavy side computationally, keeping track of multiple potential stationary objects, each with a classification vote not to mention a historic baseline. All this to say that the computational load was too much for the disruptive video 10. It repeatedly crashed my program, and I was unable to develop a solution to this problem before the deadline. Even without video 10, this solution was more successful than the others, hence I opted to submit it. If I had more time, I would attempt to decrease the computational complexity of my solution.



This picture was taken on a previous version but illustrates my point well. You won't be able to recreate it using the code I have submitted.

Results analysis:

Before I get into the results, I would like to mention that while I was unable to evaluate the video with my solution, I have accounted for this failure within my evaluation.

In order to evaluate the performance of the static object detection and the classification individually, I added a counter that would increase if the detections ever lined up with the ground truth. This simple evaluation method didn't take into account false positives, but gave me a rough idea of how each section was performing, paired of course with the visual output. These results reinforced what has been established in the visual analysis and will be reinforced below. It's worth noting that these numbers don't take video 10 into account. Setting the video 10 values to 0 (since it doesn't run) would yield 0.64 and 0.43 respectively. Interestingly these numbers are quite close to the recall and precision below, perhaps suggesting that recall reflects the static object detection and precision the object classification.

```
COUNTER-BASED EVALUATION
Static Object Detection: 9/12 (75.0%)
Correct Classification: 6/12 (50.0%)
```

As well as the classification and stationary object identification, I also designed evaluation characteristics for my solution. As seen below I used an F1 score to numerically determine how effective my solution was. At 0.364 there is definite room for improvement. Above I have extensively explained the shortcomings of my solution, and they account for this result.

```
OVERALL RESULTS
Total TP: 6, Total FP: 17, Total FN: 4
Overall Precision: 0.261
Overall Recall: 0.600
Overall F1 Score: 0.364
PS C:\Users\orarp\Documents\Year 4\Computer Vision\Assignment> █
```

The recall value is the best of the 3 metrics across the videos. This was not surprising however it is slightly lower than I expected. My solution occasionally found the object but misclassified it. These scores would have brought the recall down slightly.

The precision was low, but I expected this based on what I saw. While I had success reducing false positives, I found them impossible to eliminate and my solution reflects this. The nature of the task, favours recall over precision. With bomb scares or theft

(applications of the assignment) false positives are much preferable to false negatives. While my solution has both it certainly performs better on the recall side.

The F1 score provides rounded view of how my solution performs. As I mentioned the score of 0.364 is below average and less than satisfactory. There are several approaches I would take to improve this result if I were reattempting the assignment. The first thing I would do is fine tune the classification. I would experiment with different techniques and different combinations of techniques to correctly classify stationary objects.

The next thing I would do is experiment with the stationary object detection to detect repeat events starting with GMM. I would experiment with different learning rates, looking at the pros and cons of each.

The final thing I would do after looking into the above, is reduce the computational complexity of my solution. This would help with edge cases such as video 10 where there is a lot of interference and disruption.

These changes would by no means result in a perfect application, however I expect they would improve both my f1 and precision score.

Conclusion:

To recap, based on my evaluation, my solution certainly has shortcomings, even failing completely under certain scenarios. That said visually it looks promising. I believe my method and use of computer vision techniques has merit and could produce a fully functioning application after some minor tweaks. Thank you for your attention.